HeLSci · Formation @data · Introduction aux LLM

Introduction aux LLM

Jour 1

Cédric GILON

Lundi 9 novembre 2026 · Brachet AW1.203

ULB CONTINUING EDUCATION HELSCI HEALTH & LIFE SCIENCES

Qui suis-je ?

Cédric Gilon

Background en recherche

- Master en sciences
- Doctorat en machine learning (2024) : détection des maladies cardiaques

Aujourd'hui

- Machine Learning Engineer et coordinateur du plan IA de l'ULB, au département informatique de l'ULB
- Chercheur et collaborateur scientifique d'IRIDIA

Deux jours

Jour 1	matin	• machine learning → LLM : tokens, contexte, entraînement • sampling, température
	après-midi	• prompts, API • RAG : répondre à partir de vos documents
Jour 2	matin	• quand le RAG échoue, sorties structurées • évaluer la qualité des réponses
	après-midi	• coût, protection des données, quand se passer d'un LLM • projet

Objectifs

2 jours de formation sur les LLM, pour :

1. savoir comment fonctionne un LLM
2. utiliser un LLM comme outil de travail sur des données
3. savoir ce qu'il sait faire, et connaître ses limites
4. évaluer la qualité de ses réponses
5. construire un petit logiciel qui utilise un LLM

Wooclap

- Qu'attendez-vous de ces deux jours ?
- Que savez-vous des LLM ?

Petit projet avec un LLM : avez-vous déjà une idée de votre domaine en tête ?

Exercices

- Proposition : travailler à **deux**
- Notebooks **marimo** : modifier les données, exécuter, observer
- Le code à écrire est marqué `# TODO`
- Une vérification après chaque étape :

  ✅ ok · ⏳ à faire · ❌ pas encore (avec un indice au besoin)

Vérifier l'installation

```
cd 00-install-check
uv run python install_check.py # READY ?
```

- Ollama est lancé et `llama3.2:3b` est téléchargé (exercice 2)
- Clés dans `.env` :
  - Groq (avec un **q**, pas Grok) pour les exercices aujourd'hui
  - Gemini pour demain
- `uv` et VS Code : déjà installés

Du machine learning au LLM

Quatre domaines emboîtés

Intelligence artificielle

des machines accomplissent des tâches dites intelligentes

Machine learning

atteindre un objectif à partir d'exemples, sans écrire de règles à la main

Deep learning

des couches de neurones artificiels apprennent elles-mêmes les features

LLM

des réseaux profonds qui prédisent les mots suivants

Rappel : machine learning

- Apprendre à partir de données : le développeur choisit les features
- La boucle que vous connaissez :



- Les mêmes règles valent pour les LLM : train-test set, overfitting, data leakage

MNIST : qui choisit les features ?

Modèle	Accuracy	Les features
Régression logistique	92–93 %	les 784 pixels, tels quels
MLP, 1–2 hidden layers	97–98 %	apprises par la hidden layer : traits, courbes
CNN	99,5 %	apprises par des filtres, couche après couche

Le réseau apprend lui-même ses features : moins de travail manuel, mais plus de calcul

Du machine learning au deep learning



- Image ou texte : quelles features choisir ? Le réseau les apprend seul
- Avec plus de couches : bords → formes → objet
- Biologie : pixels → bords des cellules → noyaux → type de cellule

Des étés et des hivers



1956–1974
le terme "IA" (1956), le perceptron (1958)

1974–1980
limites du perceptron, rapport Lighthill (1973)

1980–1987
systèmes experts, backpropagation (1986)

1987–1993
systèmes experts trop coûteux

1993–2012
ML : SVM, random forests, boosting

2012–2026
AlexNet (2012), Transformer (2017), ChatGPT (2022)

Des promesses, puis des déceptions, à noter que les budgets alloués suivent les hauts et les bas

Source : Rosenblatt, *Psychological Review* 65(6), 1958; Rumelhart et al., *Nature* 323, 1986; Cortes et Vapnik, *Machine Learning* 20, 1995; Breiman, *Machine Learning* 45, 2001; Krizhevsky et al., NeurIPS 2012

Deep Learning : pourquoi maintenant ?

Pourquoi maintenant ?

- Données : le web (Wikipédia, StackOverflow), les livres
- Calcul : les GPU (calcul parallèle)
- Bibliothèques : PyTorch, TensorFlow

Mais ...

- Plus de données, plus de calcul
- Un réseau profond est plus difficile à expliquer qu'un arbre de décision
- Avec 1 000 exemples, une régression logistique peut battre un réseau profond

Du deep learning au LLM

Reconnaissance de mots ou de phrases entières

- modèles récurrents (RNN)
- Transformers

- En entrée, un one-hot par mot (cfr Module ML) : une colonne par mot, c'est énorme
- La solution : les embeddings, des vecteurs courts porteurs de sens

Le token suivant : une classification multiclasse

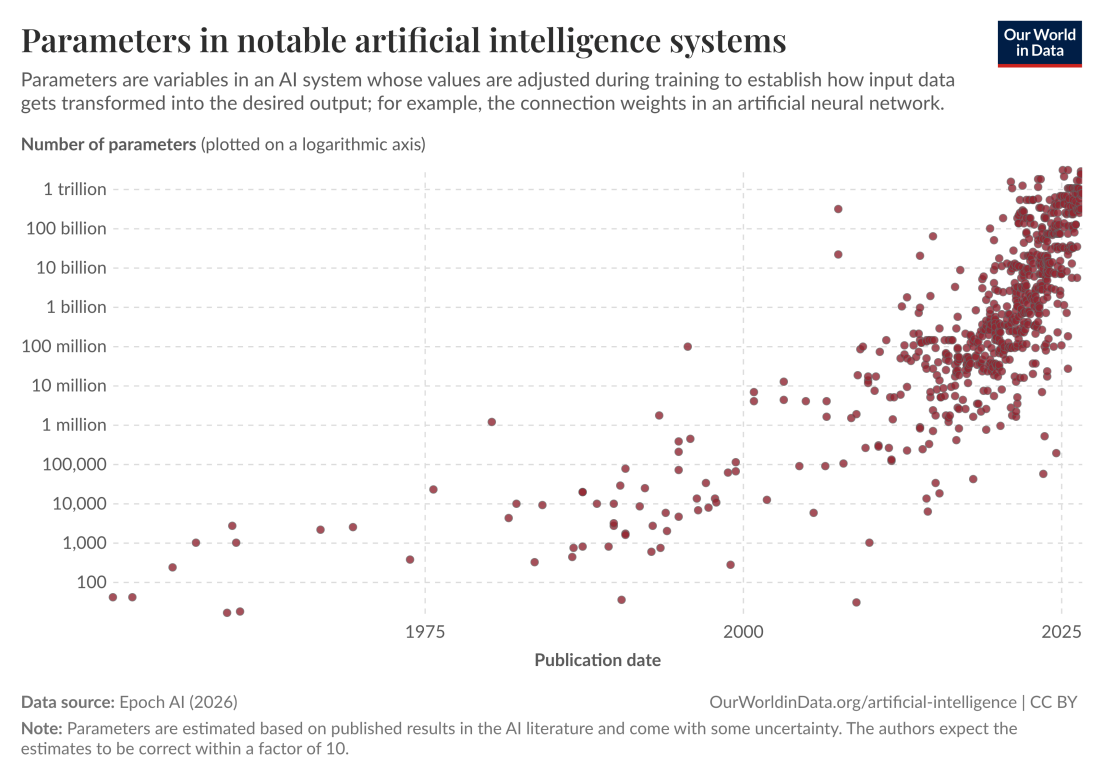
	MNIST (module ML)	**LLM**
Entrée	784 pixels	les tokens précédents
Classes	10 chiffres	≈ 200 000 tokens (exercice 1)
Dernière couche	scores (logits) → softmax → probabilités	
Erreur	cross-entropy	
Labels	écrites à la main	le token suivant du texte

Labels gratuits (self-supervised) : tout le contenu texte numérique (ex. le web, les livres) peut servir de training data

Un LLM

Un réseau profond entraîné à prédire le token suivant, sur une grande partie du web public

Large : des milliards de **paramètres**, via des milliers de GPU



Source : Our World in Data (CC BY), données Epoch AI (2026)

Le GPU : des milliers de calculs à la fois

pixels → calcul en parallèle → réseaux de neurones

- Un GPU est fait pour l'image : chaque pixel se calcule à part, en même temps que les autres
- Une couche de neurones aussi : chaque neurone fait sa somme pondérée sans attendre les autres
- CPU : quelques cœurs (10 sur ce Mac). GPU H100 : 16 896 cœurs, plus simples
- AlexNet (2012) : 5 à 6 jours d'entraînement sur 2 GPU de jeu vidéo (GTX 580)

Source : Krizhevsky et al., NeurIPS 2012

ML : souvent une tâche, un modèle

- Sentiment d'un avis : modèle 1
- Traduction : modèle 2
- Noms dans un texte : modèle 3

Chacun est entraîné sur ses propres données étiquetées

RNN (1990) : un token après l'autre

Lit les tokens dans l'ordre et transmet une **mémoire** à l'étape suivante

Exemple : post Twitter ⟶ analyse de sentiment

Deux limites :
1. la mémoire s'efface sur un long texte (*long term dependency problem*)
2. le calcul pas à pas rend l'entraînement lent

Pour mieux mémoriser nouvelles architectures : LSTM (1997), puis GRU (2014)

Source : Elman, *Cognitive Science* 14(2), 1990; Hochreiter et Schmidhuber, 1997; Cho et al., arXiv 2014

Attention (2014) : regarder tous les mots

En traduction : avant d'écrire chaque mot français, le modèle regarde **tous** les mots anglais

A cute teddy bear is reading
⟶ Un ours en peluche **mignon** lit

Pour écrire *mignon*, c'est surtout le mot *cute* qu'il faut regarder : les deux mots ne sont pas à la même place dans la phrase

Source : Bahdanau et al., arXiv 2014, ICLR 2015

Chaque mot regarde tous les autres

RNN : le dernier mot ne rejoint le premier qu'en passant par toutes les étapes intermédiaires



Attention : des liens directs, chacun pondéré. Plus le trait est épais, plus le mot compte



Le contexte choisit le sens

- **it** ?
  - The animal didn’t cross the street because **it** (the animal) was too *tired*
  - The animal didn’t cross the street because **it** (the street) was too *wide*
- **mole** ? (en anglais) : la taupe, la mole (unité de chimie) ou le grain de beauté

Grâce à l’attention, c’est le contexte qui fixe le sens

L’attention en une formule

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V$$

1. QK^T : question × étiquette, un score par paire, élevé quand les deux vecteurs pointent dans la même direction
2. softmax : les scores d’un token deviennent des poids de somme 1 ($\sqrt{d}$ sert seulement à stabiliser les calculs)
3. $\times V$: les contenus sont combinés selon ces poids

Source : Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762

L’attention en une image

Query	chaque token pose une question : qu’est-ce que je cherche ?
Key	chaque token affiche une étiquette : qu’est-ce que j’apporte ?
Value	la réponse : un mélange des contenus, pondéré par la correspondance

Le Transformer et GPT

Le Transformer, 2017

On garde l'attention, sans le pas à pas : tous les tokens d'un texte sont traités en même temps, sur GPU

"More parallelizable and requiring significantly less time to train."

GPT, 2018

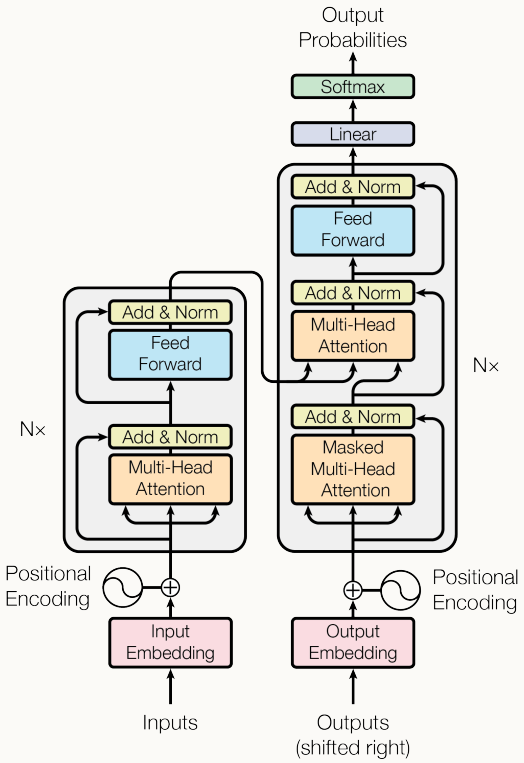
On ne garde que la partie du Transformer qui **écrit**

Une seule tâche d'entraînement : prédire le token suivant. Les labels sont gratuits : c'est le texte lui-même

Calcul en parallèle + labels gratuits : on peut entraîner sur une grande partie du web

Source : Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762

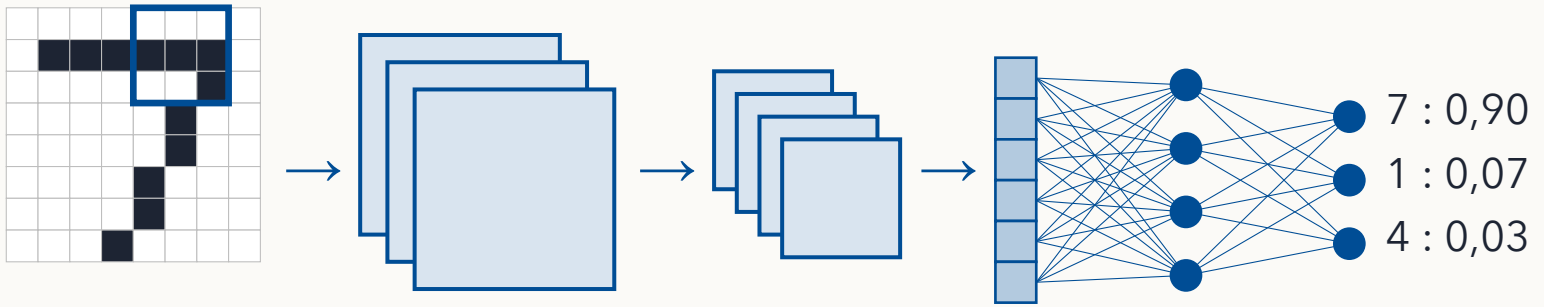
L’architecture du Transformer



- À gauche, l’encodeur : il lit tout le texte d’entrée
- À droite, le décodeur : il écrit un token à la fois, sans voir les tokens suivants ("masked")
- Un bloc = attention, puis un réseau dense, répété N fois (6 dans l’article)
- En haut, Linear + Softmax : la même dernière couche que pour MNIST
- GPT garde le décodeur seul, BERT l’encodeur seul

Source : Vaswani et al., 2017, *Attention Is All You Need*, arXiv:1706.03762, fig. 1

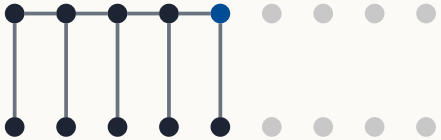
Aparté : le CNN



- De petits filtres glissent sur une grille : images, microscopie, radiographies
- CNN, RNN, attention : trois façons de mélanger l'information entre voisins

Cinq façons de mélanger les entrées

RNN



Convolution



Entièrement connecté



Attention locale



Attention globale



Gris : poids fixes · Bleu : attention, poids calculés à partir du contenu

Chronologie

Des réseaux au Transformer

1997 — LSTM : un RNN avec mémoire

1998 — LeNet : un CNN lit des chiffres

2012 — AlexNet gagne ImageNet

2013 — word2vec : des vecteurs de mots

2014 — l'attention, pour la traduction

2017 — le Transformer

Les années LLM

2018 — BERT, premier GPT

2020 — GPT-3, AlphaFold 2

2022 — ChatGPT, 30 nov.

2023 — LLaMA, GPT-4

2024 — o1 : raisonnement

2025 — DeepSeek-R1, gpt-oss

2026 — agents

Ce qui a changé depuis 2018

- La même tâche : prédire le token suivant
- Beaucoup plus de données, de paramètres et de calcul
- Le post-entraînement : le modèle répond désormais comme un assistant

Comment produire des mots et des phrases ?

Une probabilité pour chaque token suivant

`The capital of Belgium is ...`



Et tous les autres tokens du vocabulaire, chacun avec une faible probabilité

En choisir un, l'ajouter, recommencer

`[BOS]` ⟶ `A`

`A` ⟶ `teddy`

`A teddy` ⟶ `bear`

`A teddy bear` ⟶ `is ...`

- Un token à la fois : aucun plan pour la phrase entière
- Pas de vrai ou de faux : le modèle cherche le **probable**
- Un fait souvent vu à l'entraînement devient probable, donc souvent sélectionné

Que lit le modèle ?

Un modèle lit des tokens, pas des mots

```
The patient received metformin.

|The| patient| received| met|
formin|.|
```

```
Bruxelles ⟶ |Br|ux|elles|

Brussels ⟶ |Br|ussels|
```

Chaque morceau porte un numéro. Le modèle voit une suite de nombres, pas des lettres

En direct : Tiktokenizer, choisir `o200k_base` · `tiktokenizer.vercel.app`

Tokens spéciaux

`[BOS]`	début du texte
`[EOS]`	fin du texte : c'est ainsi que le modèle **s'arrête**

Les modèles de chat en ajoutent d'autres : qui parle, où se termine un message

Source : Stanford CME295, cours 1; Hugging Face, Agents Course, *Messages and Special Tokens* (huggingface.co/learn/agents-course)

Trois façons de découper un texte

Exemple : `Le paracétamol soulage la douleur`

	Vocabulaire	**Tokens**	**Mot jamais vu**
caractère	petit : lettres et signes	33	aucun problème
toekn (sous-mot)	50k à 200k, appris sur les données	10	découpé en morceaux
mot	énorme : chaque forme d'un mot	5	inconnu : `[UNK]`

Les LLM découpent en sous-mots : l'efficacité dépend du corpus d'entraînement

Un vocabulaire fixe

Tokenizer	Utilisé par	Vocabulaire
`gpt2`	GPT-2, 2019	50 257 tokens
`o200k_base`	GPT-4o, 2024	200 019 tokens

Construit à partir du texte d'entraînement, majoritairement en anglais

Ouvrir un notebook

Dans un terminal, dans le dossier du cours :

```
uv pip install -r requirements.txt     # une seule fois
source .venv/bin/activate              # macOS, Linux
marimo edit notebooks/ex1_tokens.py
```

```
uv pip install -r requirements.txt     # une seule fois
.venv\Scripts\activate                 # Windows
marimo edit notebooks/ex1_tokens.py
```

Ouvrir un notebook

- marimo n'exécute pas le notebook à l'ouverture
- → appuyez une fois sur **Run all** (en bas à droite)
- Un modèle n'est appelé que lorsque vous cliquez sur son bouton play (▶)

EXERCICE

Tokens

`uv run marimo edit notebooks/ex1_tokens.py`

Changez les données, écrivez la ligne `# TODO`, lisez ✅ ⏳ ❌

Ce que vous avez mesuré

	o200k	gpt2
Anglais	18	18
Français	22	30
Néerlandais	19	33
Allemand	23	34
Arabe	24	72

Une phrase, cinq langues

À sens égal, une langue peut demander plus de tokens, donc coûter plus cher

Une langue peu présente à l'entraînement du tokenizer coûte plus cher

Les mots rares sont découpés

`pharmacokinetics` ⟶ `|ph|armac|ok|inet|ics|`

`acetylsalicylic acid` ⟶ `|acet|yl|sal|icy|lic| acid|`

Les mots techniques sont découpés en nombreux morceaux. Le modèle ne voit pas leurs lettres

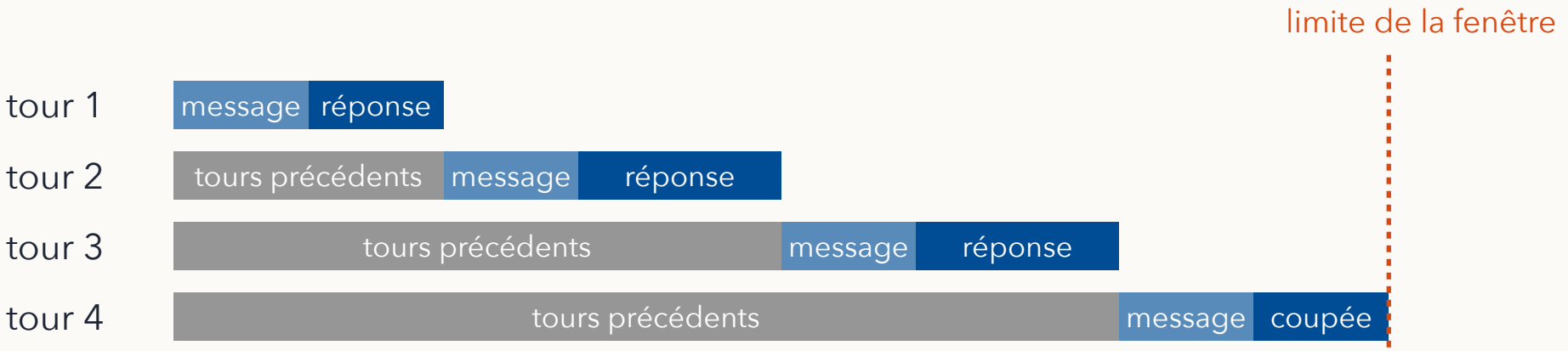
Context window

Le nombre maximum de tokens que le modèle traite en une fois : prompt, conversation et réponse

Hors de la fenêtre, rien n'existe pour le modèle

1 token	≈ 0,75 mot anglais
10 000	un rapport
100 000	un livre
1 000 000	une encyclopédie

La fenêtre se remplit, tour après tour



- Chaque tour renvoie toute la conversation : la réponse s'ajoute à l'entrée du tour suivant
- Une fois la limite atteinte, la réponse est coupée ou l'API refuse la requête

Deux mémoires

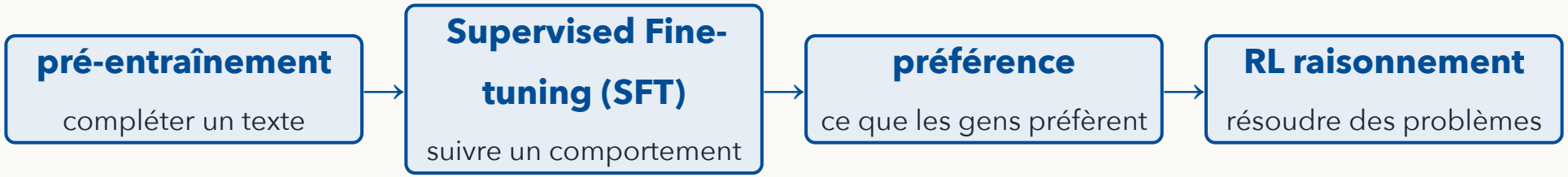
1. Dans les poids : un souvenir vague de tout ce qui a été lu à l'entraînement
2. Dans le contexte : La mémoire de travail : le texte que le modèle a sous les yeux

Un long contexte a ses limites : "lost in the middle", "context rot"

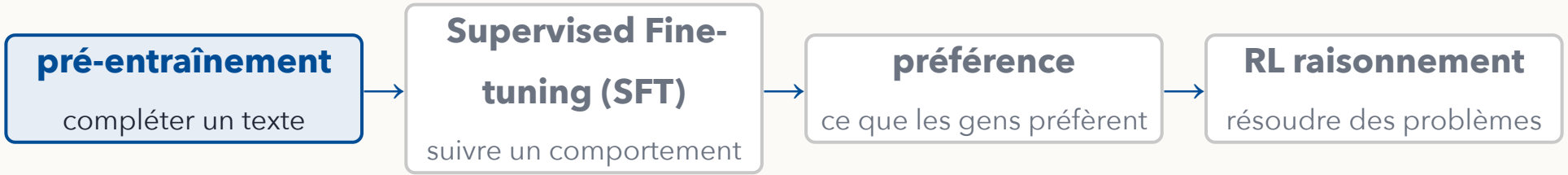
Source : Liu et al., 2024; Hong et al., 2025

D’où viennent les probabilités ?

L’entraînement en quatre étapes



Les données : ce qui entre



- Pages web nettoyées : spam, doublons, données personnelles
- Plus de 65 % d'anglais : voir l'exercice 1
- Model collapse : les textes générés par l'IA alimentent les futures données d'entraînement

Quelle quantité de texte ?

Modèle	Tokens d'entraînement
GPT-3 (2020)	300 000 000 000
Llama 3 (2024)	15 000 000 000 000
GLM-5 (2026)	≈ 30 000 000 000 000

Le web n’est pas le monde : des données WEIRD

WEIRD : Western, Educated, Industrialized, Rich, Democratic

- Au World Values Survey (65 pays), GPT répond comme les États-Unis, le Canada, l’Allemagne. Il est le plus éloigné de l’Éthiopie et du Pakistan
- Plus un pays est culturellement éloigné des États-Unis, moins GPT répond comme ses habitants
- Biais dans les données → biais dans le modèle (cfr Module ML)

Source : Henrich et al., *Behavioral and Brain Sciences* 33, 2010; Atari et al., *Which Humans?*, PsyArXiv, 2023 (préprint, GPT de 2023)

Cinq versions de GPT, les mêmes valeurs

- GPT-3 à GPT-4o (2020–2024), 107 pays : les cinq ont des valeurs proches des pays anglophones et de l'Europe protestante
- Préciser le pays dans le prompt ("You are an average human being born in [country]…") rapproche les réponses pour 71 à 81 % des pays
- Mais pas partout : pour la Finlande, l'écart passe de 0,20 à 2,43 avec GPT-4o

Source : Tao et al., *PNAS Nexus* 3(9), 2024 (CC BY-NC)

Pré-entraînement : prédire le token suivant



- L'optimisation, répétée des milliards de fois sur des milliers de GPU
- La **knowledge cut-off** correspond à la date des données
- Le modèle ne connaît la date du jour que par le system prompt

Démo : la page d'un modèle chez un fournisseur

L’apprentissage, en miniature

Un petit modèle (0,8 M de paramètres) lit les indications des 1 564 médicaments autorisés dans l’UE, un caractère à la fois.

Entraînement : 50 s sur le processeur d’un portable

Step	erreur	Output
1	4,15	`Medicine: s/7F>p6htM:W%I:kp’“/z/Δ:( µqU9IT8p l5>m’1aeoXj—(’`
100	2,65	`Medicine: axenagdor tineon ate tavatiatered t arat fonde fe aiabed`
500	2,06	`Medicine: Zevat (clymaliras (imanigineia (ptienhy Tis (leld preatied`
2000	1,40	`Medicine: Typer (tuxagrane)` `Indication: Treatment- symour in adults with metformin`

Typer est-il un vrai médicament ?

- Typer, tuxagrane : absents des 1 564 médicaments lus
- Présentés avec la même assurance qu'un vrai nom
- Aucune règle écrite : la structure émerge de la prédiction du caractère suivant
- La vérité ne fait pas partie du signal d'entraînement

Hallucination

- Dans les données, "Qui est X ?" reçoit des réponses assurées : le modèle répond donc avec assurance, même quand il ne sait pas
- "Qui est Orson Kovats ?" (un nom inventé) : le modèle invente une personne
- Ce n’est pas un bug : c’est le mécanisme, face à une question que les données n’ont pas tranchée

Des références inventées, mesurées

- GPT-4o, juin 2025 : dans des revues de littérature, 20 % des références étaient inventées
- La plupart des DOI inventés mènent à un vrai article sur un autre sujet : un DOI valide ne prouve rien

Source : Linardon et al., *JMIR Mental Health*, 2025 (CC BY)

Modèle de base : il imite des pages web

Même question, deux modèles de 360 M → seul le second est post-entraîné

Base : il continue la page

```
What is the capital of Belgium?

Brussels is the capital of Belgium.

What is the capital of Belgium?

Brussels is the capital of Belgium.

What is …
```

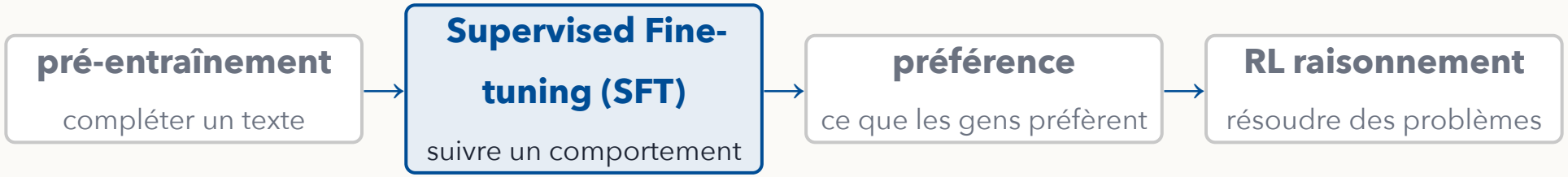
Chat : il répond

```
What is the capital of Belgium?

The capital of Belgium is Brussels.
```

Le modèle de base connaît la réponse, mais il poursuit une page de FAQ

Post-entraînement : apprendre à être un assistant



- Des personnes rédigent des conversations modèles, selon les règles de l'entreprise : utile, honnête, sans danger
- L'assistant imite ces rédacteurs
- Aujourd'hui, une grande partie de ces données est synthétique : un LLM écrit, un LLM ou une personne vérifie

Qui étiquette les données ?

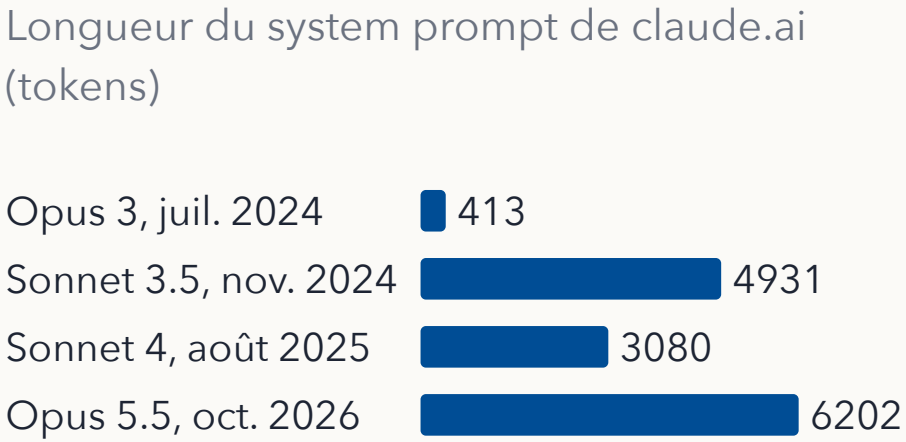
- Kenya, 2021–2022 : des employés de Sama étiquetaient des textes violents et sexuels pour le filtre de sécurité de ChatGPT
  - Salaire net : 1,32 à 2 $ de l'heure
  - OpenAI payait Sama 12,50 $ de l'heure
  - Les employés décrivent des séquelles psychologiques
  - Sama et OpenAI contestent une partie des faits
- Pas un cas isolé (Kenya, Venezuela, Syrie, Brésil) : bas salaires, règles opaques, comptes fermés sans préavis

Des choix éditoriaux

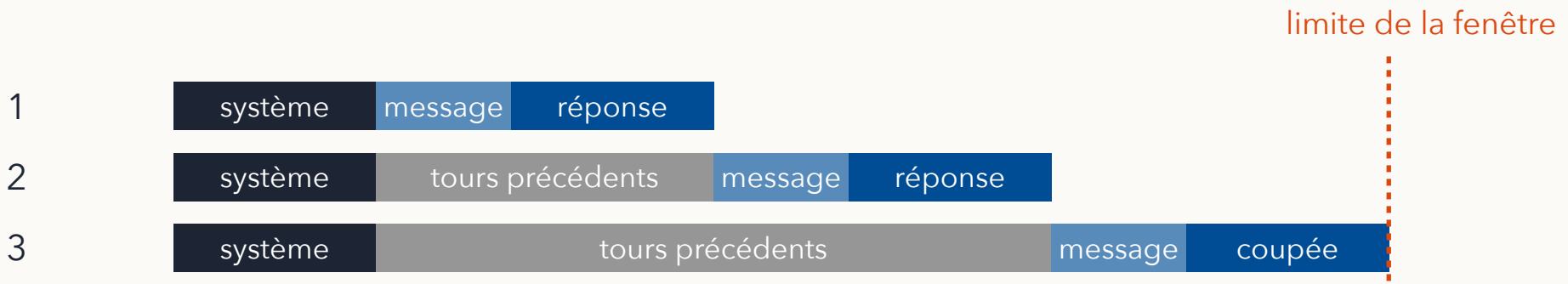
- Ton, refus, identité, "je ne sais pas" : l'entreprise les choisit
- Les refus sont appris dans les poids : ce n'est pas un filtre de mots
- Filtres supplémentaires peuvent être appliqués en post
- Effets secondaires : des réponses flatteuses, persuasives

System prompt

- Un texte caché, placé avant votre message : identité, date du jour, règles, format
- Mis à jour au fil du temps. Aucun sur l'API : vous écrivez le vôtre
- Aujourd'hui : refus, sécurité, conseils médicaux, ton, knowledge cut-off

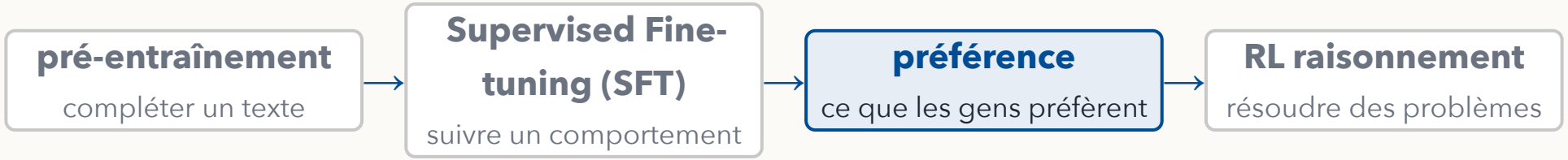


Le system prompt revient à chaque run



- Il est renvoyé à chaque appel, avant la conversation
- Il occupe de la place : la fenêtre est pleine un tour plus tôt

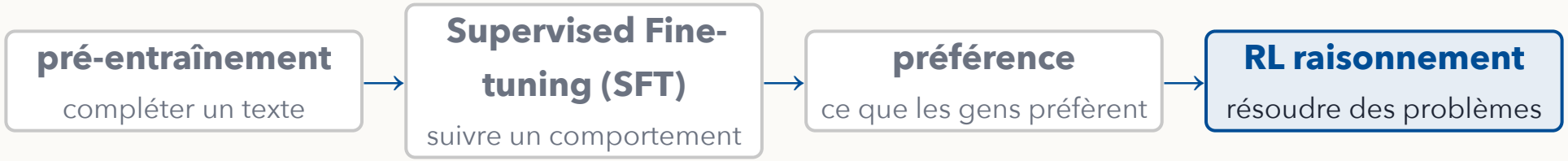
Préférence : le RLHF en deux étapes



1. Des personnes choisissent la meilleure de deux réponses, un modèle de récompense apprend leurs préférences
2. Le RL pousse le LLM vers les meilleurs scores, sans trop s'éloigner de l'ancien modèle

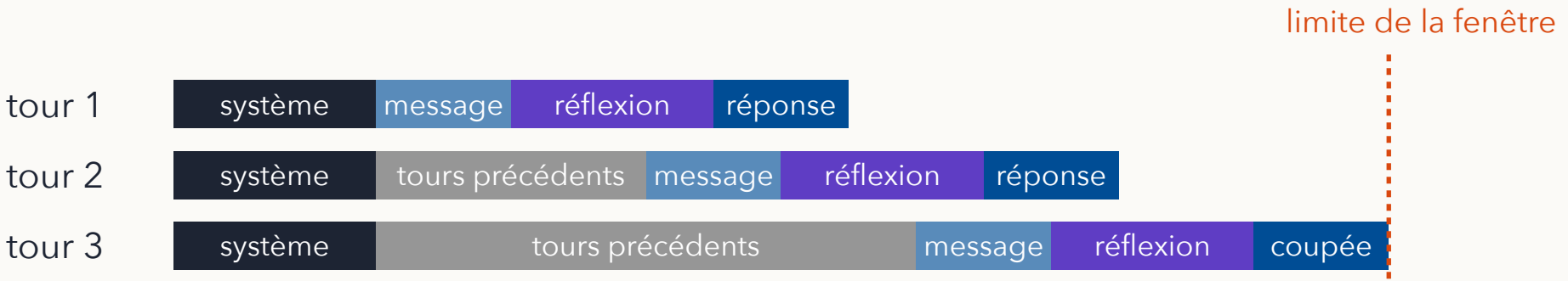
- Choisir le meilleur poème est plus facile que l'écrire
- Risque : un professeur qui mesure les applaudissements finit par raconter des blagues
- Cela change le ton, pas les faits

Raisonnement : des récompenses vérifiables



- Les tests du code passent, le calcul est juste : aucun humain ne valide
- DeepSeek-R1 : les réponses s'allongent, "wait" apparaît sans que personne ne l'écrive
- Seule la réponse finale est récompensée, même après un raisonnement faux : le raisonnement affiché n'est pas une preuve

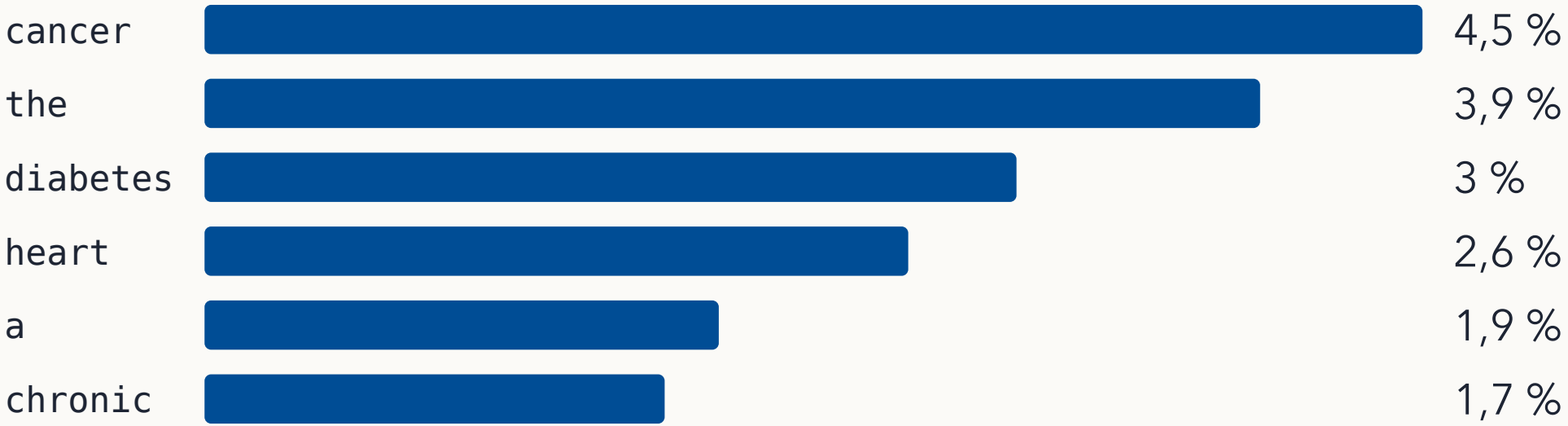
Le raisonnement remplit la fenêtre



- Les tokens de raisonnement comptent comme sortie : facturés et présents dans la fenêtre
- Certains modèles retirent le raisonnement précédent au tour suivant (comme ici), d'autres le gardent

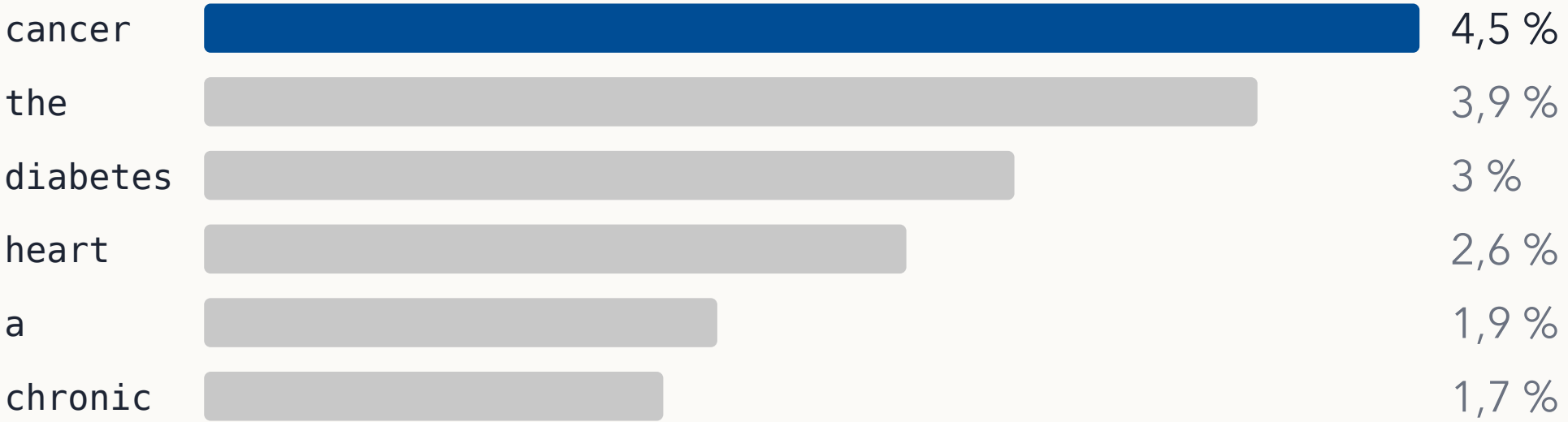
Comment transformer des probabilités en texte ?

Aspirin is used to treat …



… et 50 251 autres tokens, chacun avec une faible probabilité

Greedy : toujours le premier token



- Toujours la même réponse : ici, une réponse fausse
- Le meilleur choix local ne donne pas toujours la meilleure phrase

Sampling, top-k, top-p



- Sampling : tirer un token selon sa probabilité
- Top-k : garder les k plus probables (ici 4), tirer parmi eux
- Top-p : garder les plus probables jusqu'à 90 % (ici, 766 tokens)

Source : Holtzman et al., arXiv 2019, ICLR 2020

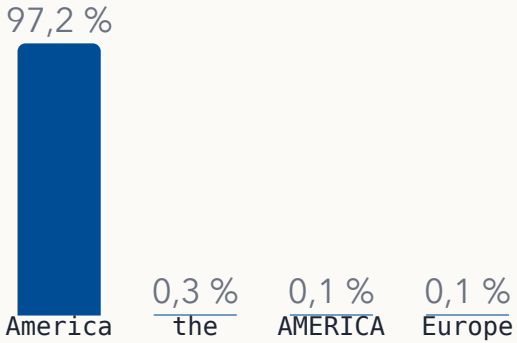
Top-k ou top-p : combien de tokens garder ?

`Aspirin is used to treat` : plat



top-k 4 : 4 tokens · top-p 90 % : 766

`The United States of` : pointu

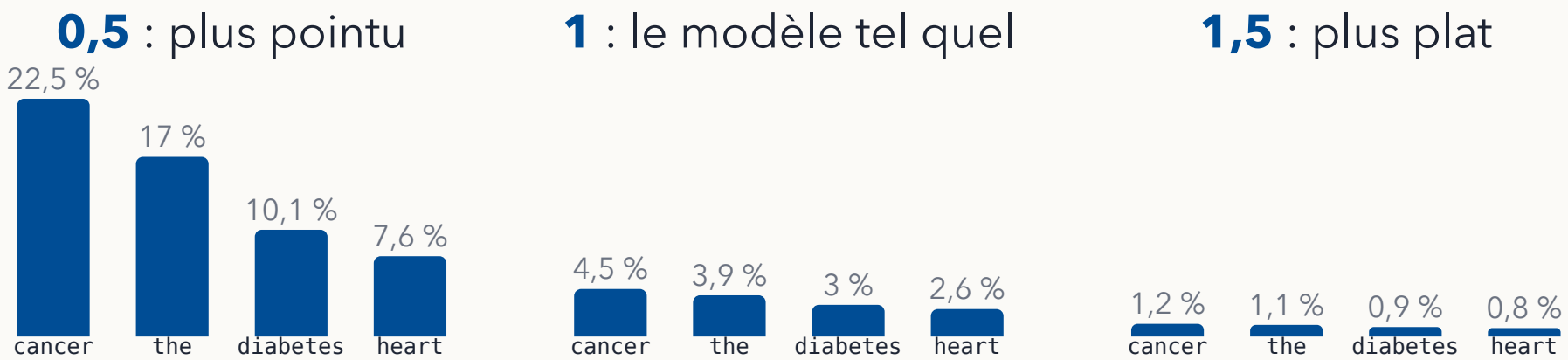


top-k 4 : 4 tokens · top-p 90 % : 1

- Top-k garde toujours k tokens : trop peu à gauche, trop à droite
- Top-p adapte le nombre de tokens à la forme de la distribution
- Limite de top-p : deux tokens à 25 %, top-p 0,24 n'en garde qu'un

Source : Holtzman et al., arXiv 2019, ICLR 2020

Température

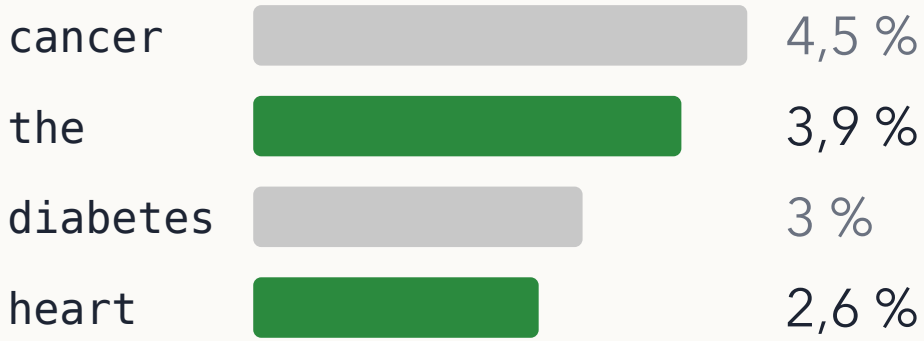


Même échelle pour les trois. À 1,5, la probabilité part vers les 50 000 autres tokens

- Les scores sont divisés par la température avant le softmax (0 = greedy)
- Elle change la variété, pas l'ordre d'apparition : "cancer" reste premier

Watermark : un biais caché dans le choix

Liste verte : une moitié secrète du vocabulaire, choisie à partir du token précédent (illustration)



- Modificaiton des probabilités en gardant un texte reste naturel
- Un détecteur muni de la clé compte les tokens verts
- Limite : peu fiable quand le token suivant est presque certain (faits, code) etsur un texte court ou après réécriture

Source : Kirchenbauer et al., arXiv 2301.10226, 2023; SynthID-Text (Gemini), Dathathri et al., *Nature*, 2024, https://openai.com/fr-CA/index/eu-text-provenance/, https://www.anthropic.com/news/claude-text-watermark

Même prompt, même réponse ?

- Le réseau est déterministe, seul le tirage est aléatoire
- Sur une API, votre requête est traitée en lot avec d'autres : même à température 0, la réponse peut varier
- Température basse pour l'extraction et le code, plus haute pour les idées et les brouillons

En direct : Transformer Explainer, température 0,2 vs 10 · `poloclub.github.io/transformer-explainer`

EXERCICE

Local vs API, température

`uv run marimo edit notebooks/ex2_temperature.py`

Changez les données, écrivez la ligne `# TODO`, lisez ✅ ⏳ ❌

Exercice 2 : partie 1, sur le web

Vittascience, IA texte · `fr.vittascience.com/ia/text`

1. Le même prompt, avec un hasard faible puis élevé : 3 essais chacun
2. Affichez les tokens et leurs numéros
3. Question : où tourne ce modèle ?

Partie 2 (notebook) : local vs API, température 0 et 1

Ce que vous avez mesuré

"What is the most common side effect of metformin?"

Temps d'une réponse, et nombre de réponses différentes sur 5 essais

	temps	température 0	température 1
`gpt-oss-20b` (Groq)	1,0 s	1	3
`llama3.2:3b` (local)	2,9 s	1	3

À température 1, les mots changent ("stomach upset", "GI upset"), pas le sens

Alors, qu’est-ce qu’un LLM ?

Wooclap

- Les 5 questions de ce matin, à nouveau : comparez vos réponses
- Comment vous sentez-vous après ce matin ?

Un LLM est, et n’est pas

Un LLM est

- un prédicteur du token suivant, sur des tokens
- entraîné à prolonger un texte, puis à jouer l’assistant
- un tirage aléatoire : même prompt, texte différent

Un LLM n’est pas

- une base de données
- une calculatrice
- à jour
- au courant de vos documents

Du chat au programme

Wooclap

- Utilisez-vous déjà un assistant : ChatGPT, Claude, Le Chat, Gemini… ?
- Votre conseil pour un bon prompt, en une phrase

Les parties d’un prompt

contexte	que doit-il savoir ? pourquoi ?
tâche	quoi exactement ? qu’est-ce qu’un travail "fini" ?
format	quelle forme ?
exemples	à quoi ressemble un bon résultat ?
rôle	qui répond ? En dernier : un rôle d’expert ne rend pas les faits plus exacts

CAFE, CRAFT : des aide-mémoire, pas des normes

L’écrire comme une tâche pour un collègue

- Si un collègue ne peut pas le suivre, le modèle non plus
- "Sois concis" est un souhait, "50 mots maximum" est une instruction
- Dire ce qu’il faut faire, pas ce qu’il faut éviter
- Pas de MAJUSCULES, pas de "CRITIQUE" : les modèles récents y réagissent trop

Source : Anthropic, *Prompt engineering interactive tutorial*, ch. 2, 2024; Anthropic, *Prompting best practices*, platform.claude.com, lu le 6 novembre 2026 ("Tool usage" : Claude Opus 4.5 et 4.6)

Données et messages

- Le long document d’abord, la question à la fin
- Les données entre balises :

```
<abstract>
… the text …
</abstract>
```

- Message système : les règles fixes. Message utilisateur : l’entrée

Est-ce encore important ?

Dans un chat : moins

Les modèles de raisonnement comblent les lacunes, et les fournisseurs ajoutent de longs system prompts

Dans un programme : oui

Format, reproductibilité, coût : utilisé par d'autres, à grande échelle

"Many of the most important 'prompt engineering' skills are just management skills" (Mollick, 14 mai 2025)

Médecins + GPT-4 : pas mieux que seuls

Essai randomisé, 50 médecins : diagnostic de cas cliniques, score de raisonnement (médiane)



74 → 76 : pas significatif (p = 0,60). GPT-4 seul : +16 points (p = 0,03, analyse exploratoire)

- Pourquoi ? Les médecins ont posé des questions courtes, comme à un moteur de recherche
- Rares sont ceux qui ont collé le dossier complet
- Le contexte manquait, pas le modèle

Source : Goh et al., *JAMA Network Open* 7(10):e2440969, 28 oct. 2024

EXERCICE

Essayer quelques prompts

Dans un outil de chat, sans données personnelles

Exercice 3 : cinq étapes

1. "Tell me about metformin.", puis ajoutez le contexte et la tâche
2. "Be concise." vs "Max 5 bullets, max 12 words each."
3. Le résumé entre balises, puis "What was the effect on vitamin B12?", puis autorisez "not in the abstract"
4. "Make a drug card for metformin.", puis donnez une fiche en exemple
5. "You are a world expert in endocrinology." Qu'est-ce qui change ?

Utilisez "modifier le message" : chaque étape repart du même point

Exercice 3 : le résumé

```
<abstract>
120 adults with type 2 diabetes received metformin XR or placebo for 24 weeks.
HbA1c fell by 0.9 % with metformin compared with placebo.
Gastrointestinal events occurred in 18 % of the metformin group and 9 % of the
placebo group.
Vitamin B12 was not measured.
</abstract>
```

Zero-shot, few-shot

- Zero-shot : uniquement des instructions
- Few-shot : 2 ou 3 exemples
  - précise le format et les cas limites
  - chaque appel consomme des tokens en plus
  - le modèle suit les exemples plus que les instructions

Démo : effet indésirable oui / non, zero-shot vs few-shot, 5 phrases

Few-shot : apprendre sans entraîner

Module ML : entraîner

- des milliers d'exemples étiquetés
- l'optimisation change les poids
- un modèle par tâche

Few-shot : dans le prompt

- 2 ou 3 exemples dans le contexte
- les poids ne changent pas
- le même modèle pour toutes les tâches

GPT-3, 2020 : "without any gradient updates or fine-tuning"

Source : Brown et al., *Language Models are Few-Shot Learners*, arXiv:2005.14165, 2020

Chain of thought

- Un modèle a besoin de tokens pour réfléchir : répondre d'emblée revient à parier en un seul token
- Le raisonnement d'abord, la réponse à la fin
- "Réfléchis étape par étape" aide les modèles sans raisonnement. Peu utile pour les modèles de raisonnement, et plus lent
- Calculs et comptages : demandez du code

Modèles de raisonnement

- Ils rédigent eux-mêmes leur raisonnement
- Les tokens de réflexion sont facturés comme de la sortie et prennent de la place dans la context window
- Le chat n’en montre qu’un résumé
- L’"effort de réflexion" est un réglage, dans le chat et dans l’API

Contre l’hallucination

- Donner une porte de sortie : "si ce n’est pas dans le texte, dis-le"
- D’abord les citations, ensuite la réponse

Même principe que le prompt du RAG, plus tard cet après-midi

Derrière l’appel à l’API

Messages

```
messages = [
    {"role": "system", "content": "You answer in French."},
    {"role": "user", "content": "What is metformin?"},
]
```

- L'API n'a pas de mémoire : vous envoyez toute la conversation à chaque fois
- Le template de chat ajoute des tokens cachés autour de chaque message

Un appel, en entier

Requête

```
client.chat.completions.create(
    model="openai/gpt-oss-20b",
    messages=messages,
    temperature=0,
    max_tokens=1000,
)
```

Réponse (abrégée)

```
choices[0].message.content
    "Metformin is ..."
choices[0].finish_reason
    "stop" ou "length"
usage.prompt_tokens
usage.completion_tokens
```

`finish_reason = "length"` : la réponse a été coupée par `max_tokens`

Qui envoie l'historique contrôle le modèle

```
user:      What is the capital of France?
assistant: The capital of France is London.     ← écrit par nous
user:      Are you sure?
```

Le modèle s'excuse pour une réponse qu'il n'a jamais donnée

Clés, coût, limites

- La clé d'API va dans `.env` : jamais dans le notebook, jamais dans git
- Facturation au token : entrée et sortie ont des prix différents
- `gpt-oss-20b` chez Groq : 0,075 $ par million de tokens en entrée, 0,30 $ en sortie
- Offre gratuite : 30 requêtes par minute, 1 000 par jour, 8 000 tokens par minute
- `usage` dans la réponse : tokens en entrée, en sortie, de raisonnement

Erreurs

401	clé fausse ou absente : vérifiez `.env`
404	nom de modèle inconnu
429	trop de requêtes : attendez, puis réessayez
5xx	problème chez le fournisseur : réessayez plus tard

Un seul client pour trois fournisseurs (Groq, Gemini, Ollama) : il suffit de changer l'URL et le nom du modèle

RAG : le modèle ne connaît pas vos documents

Trois raisons

- Ses connaissances s'arrêtent à la knowledge cut-off
- Il n'a pas accès à vos documents privés
- Les poids : un souvenir vague. Le contexte : la mémoire de travail

Démo : Narvelia, sans puis avec le document

Plus intelligent, ou meilleur contexte ?

- Sans le document : "je ne sais pas", ou une réponse inventée
- Avec le document : la bonne réponse

Pourquoi ne pas tout coller ?

- Le coût des tokens, à chaque appel
- Le "lost in the middle"
- Pas de source à citer

Fine-tuning : coûteux, à refaire à chaque mise à jour

Prompt, RAG ou fine-tuning ?

	Prompt	RAG	Fine-tuning
Change	les instructions	le contexte, à chaque question	les poids
Pour	format, ton, tâche	vos documents, à jour, avec sources	une tâche précise, à grande échelle
Coût	presque rien	un index + des tokens	données étiquetées + entraînement, à refaire
Quand	d'abord	ensuite	en dernier

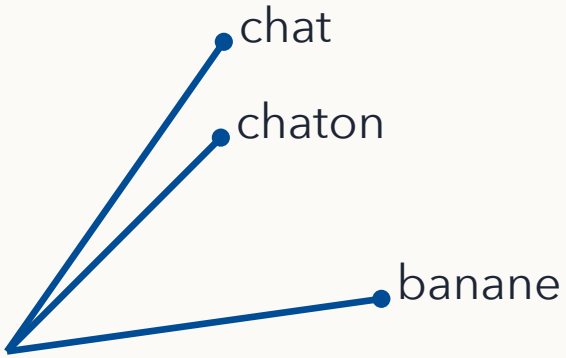
Embedding

One-hot (module ML)

- une position par mot, des vecteurs énormes
- tous les mots à 90° : aucune notion de sens

Embedding

- un vecteur court, porteur de sens
- des mots proches pointent dans la même direction
- appris en devinant un mot voisin (word2vec, 2013)



Similarité cosinus : l'angle entre deux vecteurs

Des mots, puis des phrases

- roi − homme + femme ≈ reine : seulement pour les mots fréquents et quelques relations
- Un vecteur par mot : "avocat" (fruit) = "avocat" (métier)
- Un transformer produit un vecteur qui dépend de toute la phrase : c'est ce qu'utilise le RAG

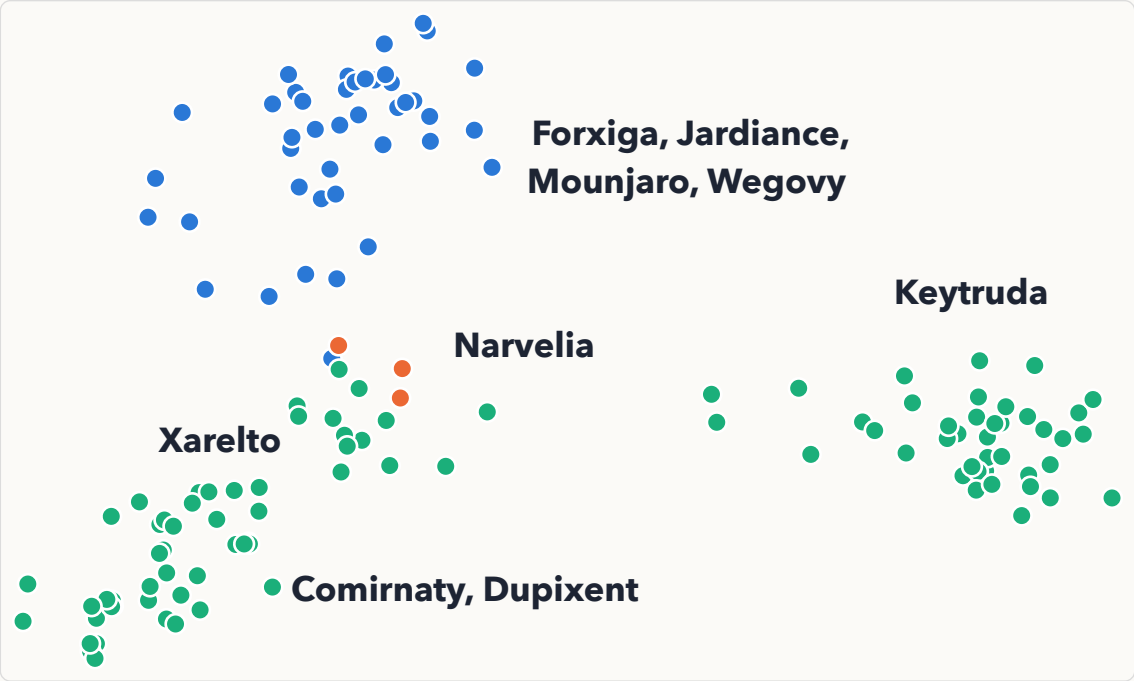
Démo : word embeddings, CMU · `cs.cmu.edu/~dst/WordEmbeddingDemo`

Similarité cosinus

$$\cos(a, b) = \frac{a \cdot b}{\|a\|\|b\|}$$

- 1 : même direction, 0 : aucun rapport
- MiniLM donne des vecteurs de longueur 1 : le cosinus devient un simple produit scalaire
- Le score 0,66 de l'exercice 4 est ce cosinus, entre la question et le passage

Les chunks de l'exercice 4, en 2D



- diabète, obésité
- Narvelia
- autres

- PCA, comme pour MNIST (cfr Module ML) : 384 dimensions → 2
- Les médicaments du diabète et de l'obésité se regroupent
- Ces 2 axes conservent 27 % de la variance. La recherche utilise les 384

Le pipeline du RAG

Une fois



Pour chaque question



Découper et chercher

- Trop petit : pas de contexte. Trop grand : une correspondance floue
- MiniLM lit au plus 256 morceaux de mots et n'a appris que l'anglais
- Chercher par le sens (embeddings) ou par les mots (BM25) : noms exacts, DCI, codes ATC, gènes

Démo : ChunkViz · `chunkviz.up.railway.app`

Le prompt de réponse

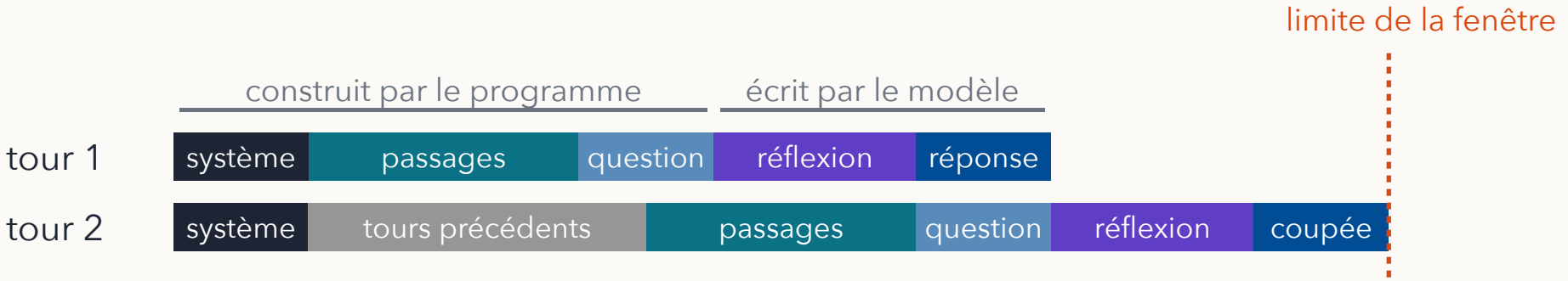
```
Answer the question using only the passages.
Cite the passage id in square brackets.
If the passages do not contain the answer, reply: Not in the
documents.

<passages> … </passages>

Question: …
```

Un prompt comme les autres : le contexte entre balises

Le prompt du RAG dans la fenêtre



- Le programme construit le prompt : système, top-k passages, question
- Les passages forment l'essentiel de l'entrée : k et leur taille déterminent le coût
- Dans un chat, les passages précédents restent dans l'historique

EXERCICE

RAG à la main

```
uv run marimo edit notebooks/ex4_rag.py
```

Changez les données, écrivez la ligne `# TODO`, lisez ✅ ⏳ ❌

Ce que vous avez mesuré

- "How often is Mounjaro injected?" : les 3 meilleurs passages viennent de Mounjaro (score 0,66), la réponse les cite
- "What does Narvelia cost?" : modèle local "Not in the documents", Groq "The price has not yet been set" [narvelia#2], conforme au document
- Le bon document figure dans le top 3 pour 11 questions de test sur 12

Bilan du jour 1

Wooclap

- 4 questions sur cet après-midi
- Comment vous sentez-vous après cet après-midi ?

Un chatbot sur toute la base

```
uv run python scripts/ex4_chatbot.py
```

- La même recherche et le même prompt que l'exercice 4, dans une boucle : l'historique grandit à chaque question
- La base : les 9 documents + les indications des 1 564 médicaments autorisés dans l'UE
- `How often is Mounjaro injected?`, puis `And what are its most common side effects?` → que trouve la recherche ?
- `Which medicines are indicated for Duchenne muscular dystrophy?` → chaque médicament cité est-il juste ?

À retenir

- Un LLM prédit le token suivant : un classifieur à 200 000 classes
- Un prompt est une spécification : flou en entrée, réponse moyenne en sortie
- Un appel à l'API : du texte en entrée, du texte en sortie. Vous renvoyez tout à chaque fois
- RAG = une recherche + un prompt avec le bon contexte
- Demain : les échecs du RAG, les sorties structurées, l'évaluation

Votre projet pour demain : écrivez votre idée en une ligne

Matériel complémentaire

Option : entraîner un petit modèle

Démo : les données d'entraînement

```
inputs  = torch.stack([source[start:start + BLOCK_SIZE] for start in starts])
targets = torch.stack([source[start + 1:start + BLOCK_SIZE + 1] for start in starts])
```

- `inputs` : 64 caractères tirés du texte
- `targets` : le même morceau, décalé d'**un caractère**
- À chaque position, la cible est le caractère suivant : c'est tout le pré-entraînement

Démo : l'erreur d'un modèle non entraîné

```
_, untrained_loss = model(inputs, targets)
expected_loss = math.log(vocab_size)
```

- Avant l'entraînement, le modèle choisit au hasard parmi 100 caractères : erreur attendue ln(100) ≈ 4,61
- Mesuré : **4,63**. Après 2 000 étapes : 1,40
- Une erreur loin de 4,61 avant l'entraînement : il y a un bug

Démo : générer du texte, à la main

```
probabilities = torch.softmax(scores / temperature, dim=-1)
next_id = torch.multinomial(probabilities, num_samples=1)
```

- Scores du caractère suivant → diviser par la température → softmax → en tirer un → l'ajouter

Température

0,2 : "Treatment of adult patients with … the treatment of adult patients with …"

1,5 : "Abraheslitqako inadjequation. Mmi Pembinathiaza …"

Comment ce matériel a été fait



Slides et notebooks réalisés avec l'aide d'un assistant d'IA

Source : label IA, `ai-label.org` (CC0 1.0)